10 preguntas: Redes Neuronales Convolucionales (CNN)

Raul Terrazas Sanchez

Pregunta 1 — Convolución a mano (3×3)

1. Tamaño de salida: ⌊(5−3)/1⌋+1 = 3, es decir, un mapa de 3×3 (9 posiciones del filtro).

2. Esquina superior izquierda. El parche es [0 0 1 / 0 1 2 / 1 2 3] y el kernel tiene [1 0 −1] en cada fila:

Fila	Producto
1	0·1 + 0·0 + 1·(−1) = −1
2	0·1 + 1·0 + 2·(−1) = −2
3	1·1 + 2·0 + 3·(−1) = −2

Suma total = −5.

3. Cada fila del kernel es (+ 0 −), así que mide izquierda menos derecha. Responde a cambios de intensidad a lo largo del eje horizontal, que es lo que define un borde vertical. Comprobación: en el centro de la imagen cada fila es simétrica (1 2 1, 2 3 2, 1 2 1), así que 1−1 = 0 y 2−2 = 0 y la respuesta es 0 (no hay borde). En la esquina dio −5, donde sí hay gradiente.

Pregunta 2 — Contar pesos

1. Modelo A: entradas = 28·28·3 = 2352, y 2352 × 64 = 150,528 pesos.

2. Modelo B: (3·3·3)·16 + 16 = 432 + 16 = 448 parámetros.

Comparación: 150,528 / 448 = 336, por lo que el MLP tiene 336 veces más parámetros.

3. La CNN reutiliza los mismos 432 pesos en todas las posiciones (pesos compartidos), así que tiene muy pocos parámetros que ajustar y cada imagen aporta evidencia a todos ellos. Con tantos parámetros, el MLP puede memorizar las imágenes de entrenamiento.

Pregunta 3 — ¿Qué cambia si meto más capas?

1. ⌈21/2⌉ × ⌈28/2⌉ = 11×14.

2. Cada filtro de la segunda Conv2D recibe como entrada los 32 mapas de rasgos de la capa anterior, no píxeles. Combina bordes en esquinas, texturas o partes de objetos, o sea, patrones más compuestos. Como una lupa mirando lo que ya vio otra lupa, cada capa trabaja con un nivel de abstracción mayor.

3. Con muchas capas y pocos datos aparece sobreajuste: el train loss baja mientras el val loss se estanca o sube, y la brecha entre ambas crece. Primera medida: data augmentation / más datos. El sobreajuste viene de que el modelo memoriza pocos ejemplos, y más variedad ataca esa causa. Dropout y menos filtros mitigan el síntoma, pero no la falta de datos.

Dato extra, parámetros del Flatten → Dense(32): con un bloque, 11·14·32 = 4,928 y 4,928·32 + 32 = 157,728. Con dos bloques (pool → 6×7), 6·7·64 = 2,688 y 2,688·32 + 32 = 86,048. Apilar un bloque reduce el tamaño del vector aplastado y los parámetros de esa Dense (casi a la mitad).

Pregunta 4 — Formas: de volumen a vector

1. 7·7·64 = 3,136.

2. 3,136·32 + 32 = 100,384 parámetros.

3. GAP da forma (64,), es decir, 64 valores (un promedio por canal). La Dense siguiente tendría 64·32 + 32 = 2,080 parámetros, unas 48 veces menos que con Flatten. Ventaja con imagen grande: si el mapa fuera 14×14×64, Flatten daría 12,544 valores (la Dense crecería a 401,440 parámetros), pero GAP seguiría dando 64. GAP no depende del tamaño de entrada.

Pregunta 5 — Softmax y el error “casi seguro”

1. argmax([0.10, 0.05, 0.85]) = índice 2, es decir, tornado.

2. Está seguro pero equivocado: asigna 0.85 a tornado y solo 0.05 a incendio, la clase correcta. La distribución no está dispersa, y la diferencia entre la clase elegida y la correcta es 0.85 − 0.05 = 0.80.

3. Será alta. La pérdida solo mira la clase correcta: −ln(0.05) ≈ 3.0. Si hubiera acertado con 0.85, sería −ln(0.85) ≈ 0.16, unas 18 veces menos. El error confiado es el que más castiga.

Pregunta 6 — Padding, stride y “se me achicó la imagen”

padding	stride	cálculo	salida (alto×ancho)
valid	1	⌊(32−5)/1⌋+1	28×28
valid	2	⌊27/2⌋+1 = 13+1	14×14
same	1	mismo N	32×32

Tres MaxPool(2×2): 32 → 16 → 8 → 4×4. Cada celda final resume un bloque de 8×8 píxeles originales ($2^3 = 8$), por lo que un detalle de 2–3 píxeles se pierde, y los objetos pequeños dejan de ser distinguibles.

Pregunta 7 — Dropout: ¿dónde y para qué?

1. Apaga cada neurona con probabilidad 0.5: sobreviven en promedio 32 × 0.5 = 16 neuronas por paso.

2. No. En predict todas las neuronas están activas, porque queremos una salida determinista y que use toda la red. Las magnitudes son compatibles porque en entrenamiento los valores que sobreviven se escalan ×2 (inverted dropout), de modo que la suma esperada es la misma.

3. No tiene sentido. El sobreajuste se acumula durante todas las épocas, no solo al final. Dropout actúa evitando que las neuronas se co-adapten mientras se aprenden los pesos. Activarlo solo en la última época no corrige lo ya memorizado y solo añade ruido a un modelo ya entrenado, y puede empeorar la validación.

Pregunta 8 — Diseño: 5 clases de animales

1. 5 neuronas con softmax. Una por clase, y las clases son mutuamente excluyentes (cada foto es un solo animal), así que softmax da probabilidades que suman 1. Con Dense(32) previa, esa capa tendría $32 \cdot 5 + 5 = 165$ parámetros.

2. Redimensionar todas las imágenes de 224×224×3 a 21×28×3 (o cambiar input_shape y reentrenar). Si no, la forma no coincide y falla. Ojo: pasar de cuadrada a 3:4 deforma ligeramente la imagen.

3. Data augmentation (giros, zoom, desplazamientos). Síntoma que lo justifica: train accuracy alta (ej. 95%) y val accuracy mucho menor (ej. 70%), con val loss subiendo. Eso indica que el modelo memoriza en lugar de generalizar.

Pregunta 9 — CNN vs MLP vs RNN (mismo problema, mala herramienta)

1. MLP. Son 3 variables numéricas independientes, sin estructura espacial ni temporal que aprovechar.

2. CNN. Es una imagen, y el color y la forma de la luz son patrones locales espaciales.

3. RNN/Transformer. Es una secuencia, y la siguiente línea depende del orden y del contexto del prefijo.

Semáforo aplastado con un MLP grande: sí puede aprender, sobre todo en una tarea tan simple como distinguir 3 colores. Pero sacrifica:

• Parámetros: una foto de 64×64×3 son 12,288 entradas, y $12{,}288 \cdot 64 = 786{,}432$ pesos en la primera capa, contra 448 de una Conv2D de 16 filtros.

• Datos: necesita muchos más ejemplos para ajustar tantos pesos.

• Invariancia: si el semáforo cambia de posición, cada píxel activa pesos distintos y debe reaprender el patrón.

Pregunta 10 — Experimento mental: mover un gato

1. El mismo kernel (mismos pesos) se desliza por toda la imagen, así que un patrón de gato produce la misma activación en otra posición. Si el gato se corre 10 píxeles, el mapa de rasgos se corre igual (y con un pool 2×2 el corrimiento es exacto: $10/2 = 5$ celdas). El MLP asocia cada peso a un píxel concreto del vector: al correr el gato, esas entradas cambian por completo y los pesos aprendidos ya no coinciden.

2. No hay garantía. La CNN es (casi) invariante a traslaciones, no a rotaciones. Por ejemplo, el kernel [1 0 −1] girado 90° detectaría bordes horizontales, o sea, es otro filtro. Haría falta

data augmentation con rotaciones y/o más datos con gatos de lado.

3. Pregunta para un compañero: “Si duplicas todos los pesos de un kernel 3×3, ¿qué pasa con la salida de la convolución y qué detecta ahora el filtro? ¿Y si cambias el signo de todos?” Respuesta esperada: la salida se duplica y detecta el mismo patrón con más intensidad; con el signo cambiado detecta el patrón invertido, es decir, la salida es -1 veces la original.